

LogGPT: Log Anomaly Detection via GPT

Xiao Han
Utah State University
Logan, UT, USA
xiao.han@usu.edu

Shuhan Yuan
Utah State University
Logan, UT, USA
Shuhan.Yuan@usu.edu

Mohamed Trabelsi
Nokia Bell Labs
Murray Hill, NJ, USA
mohamed.trabelsi@nokia-bell-labs.com

Abstract—Detecting system anomalies based on log data is important for ensuring the security and reliability of computer systems. Recently, deep learning models have been widely used for log anomaly detection. The core idea is to model the log sequences as natural language and adopt deep sequential models, such as LSTM or Transformer, to encode the normal patterns in log sequences via language modeling. However, there is a gap between language modeling and anomaly detection as the objective of training a sequential model via a language modeling loss is not directly related to anomaly detection. To fill up the gap, we propose LogGPT, a novel framework that employs GPT for log anomaly detection. LogGPT is first trained to predict the next log entry based on the preceding sequence. To further enhance the performance of LogGPT, we propose a novel reinforcement learning strategy to finetune the model specifically for the log anomaly detection task. The experimental results on three datasets show that LogGPT significantly outperforms existing state-of-the-art approaches.

Index Terms—anomaly detection, log data, generative language model

I. INTRODUCTION

Effectively detecting abnormal events in online computer systems is critical to maintaining the security and reliability of the systems. Logs, which are a fundamental component of modern computer systems, serve as a critical source of information for system monitoring, debugging, and security auditing as they record the system status, offering valuable insights into system performance and potential issues. Anomalies in log data often signify system faults, security breaches, or operational failures, making their detection a crucial task [1]–[6].

However, the task of anomaly detection in log data is challenging due to the nature of high dimensionality, large volume, and complex structure. Machine learning models have been extensively employed for anomaly detection in log data. Traditional models, such as Principal Component Analysis (PCA) [7], Isolation forest [8], and one-class Support Vector Machines (OCSVM) [9] have been widely used. However, these models often require manual feature engineering or assume linear relationships among log entries, which makes them less effective in handling the dynamic nature of log data.

Recently, deep learning models have emerged for log anomaly detection, such as LSTM-based models like DeepLog [1], LogAnomaly [10], and OC4Seq [11], and BERT-based models like LogBERT [2]. One commonly used strategy is to borrow the idea of language modeling in the natural language

processing field to capture the sequential pattern of log data. In this paper, we call this group of log anomaly detection models **log language model**-based approaches. Particularly, the log language model is first trained to predict the next or masked log entries given the normal sequences. Then, the anomalies can be detected if the observed log entry is not in the top-K list predicted by the log language model. The rationale is that if a log sequence follows normal patterns, the log language model should be able to predict the next or masked log entries. Therefore, when an observed log entry is not in the top-K list predicted by the log language model, it means that the log entry has a low ratio to be in this specific position given the context, indicating the abnormality.

Although empirical studies have demonstrated the effectiveness of leveraging language models for log anomaly detection, the current models still face some limitations. The traditional LSTM-based log language models, such as DeepLog, often fail to fully capture long-term dependencies in log sequences. Therefore, the recently developed models usually adopt the Transformer structure [12] to model the long log sequences, such as LogBERT [2]. However, the masked log language model adopted in LogBERT may not be able to capture the natural flow in log sequences. More importantly, there is a gap between log language modeling and anomaly detection. Technically, the log language model is usually trained to correctly predict the next log entry, while the current log anomaly detection models label the anomalies if the observed log entry is not in the Top-K list predicted by the log language model. In other words, there is a gap in the objective between the training phase and the testing phase for log anomaly detection.

Inspired by the training strategy for large language models, to fill up the gap, we introduce LogGPT, a novel framework for log anomaly detection that leverages the Generative Pre-trained Transformer (GPT) model. LogGPT still harnesses the power of generative log language models to capture the intricate patterns and dependencies in log data. Specifically, LogGPT is pre-trained to predict the next log entry given the preceding sequence (prompt). More importantly, we further fine-tune LogGPT via reinforcement learning. Specifically, LogGPT employs a novel reward mechanism based on whether the observed log entry is within the Top-K predicted log entries from the log language model. If the observed log entry is found within the Top-K predictions, LogGPT will receive a

positive reward; otherwise, it will receive a negative reward. Reinforced by this reward signal, we expect that for the normal sequences, LogGPT can ensure the log entry is within the Top-K predictions.

The contributions of this paper are threefold. First, we propose LogGPT, a novel framework for anomaly detection in log data, which utilizes the generative log language model to capture the patterns of normal log sequences by training to predict the next log key given the previous sequence. This novel approach effectively addresses the limitations of both traditional machine learning models and deep learning models like DeepLog [1] and LogBERT [2], providing a more robust and effective solution for log anomaly detection. Second, we introduce a Top-K reward metric specifically designed for fine-tuning the log language model for anomaly detection. This reward metric gives a positive reward if the actual log key is in the Top-K predictions, and a negative reward otherwise, thereby guiding the model to focus on the most relevant parts of the log sequence and enhancing the accuracy of anomaly detection. Third, we conduct extensive experiments to validate the effectiveness of LogGPT in detecting anomalies in log data. Experimental results demonstrate that LogGPT outperforms state-of-the-art methods, underscoring its potential as a powerful tool for anomaly detection in log data.

II. RELATED WORK

Log anomaly detection, a critical task for ensuring system security and reliability, has received extensive research. The methods for log anomaly detection can be broadly categorized into two phases: traditional machine learning models and deep learning models.

In the early phase, traditional machine-learning models were the primary tools for log anomaly detection. Models such as Principal Component Analysis (PCA) [7], Isolation forest [8], and one-class Support Vector Machines (OCSVM) [9] were commonly used. Although these models are capable of identifying outliers in the log data, these models have several limitations. First, the traditional machine learning models usually require manual feature engineering, which is labor-intensive and might not capture the complex patterns in log data. Furthermore, these models struggle with capturing complex patterns in log sequences.

The advanced deep learning models have significantly improved the performance of log anomaly detection. In particular, Long Short-Term Memory Networks (LSTMs), known for their ability to model sequential data, have proven to be effective for log anomaly detection, such as DeepLog [1] and LogAnomaly [10]. DeepLog functions by predicting the next log key based on the preceding sequence, identifying anomalies when the actual next log key significantly deviates from the prediction. On the other hand, LogAnomaly models a log stream as a natural language sequence and develops template2vec to extract the semantic information hidden in log templates. Therefore, LogAnomaly can detect both sequential and quantitative log anomalies simultaneously. However, these models come with their own set of limitations. A primary

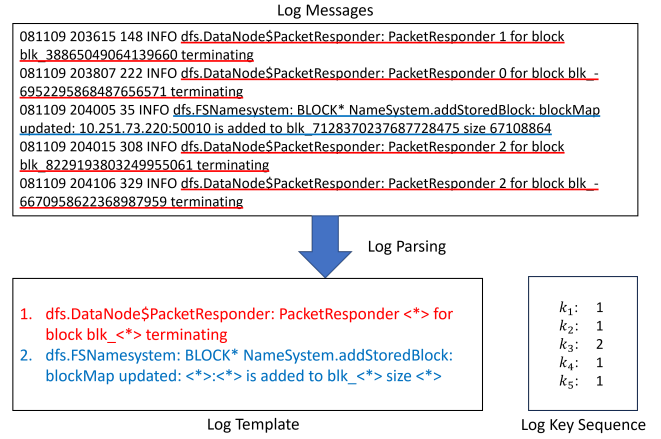


Fig. 1: Log key extraction from HDFS dataset messages via Log Parser. The message with a red/blue underscore indicates the detailed computational event for each log key separately.

challenge with LSTM is that this type of recurrent architecture struggles to encode very long or complex sequences due to its relatively simple structure. This issue is particularly pronounced in log anomaly detection, where the sequences can be quite long and complex.

To address the limitations of LSTM-based models, researchers have turned to the use of Transformer [13], which is a more powerful model to capture the long-term dependencies in the sequences, such as LogBERT [2] or CAT [14]. LogBERT is a self-supervised framework that learns the patterns of normal log sequences based on BERT [13]. Specifically, LogBERT takes normal log sequences with random masks as inputs and is trained to predict the randomly masked log entries. After training, LogBERT can encode the patterns of normal log sequences. One limitation is that the masked log language model may not always capture the natural flow of log sequences in some contexts. Moreover, the performance of LogBERT is sensitive to the mask ratio, a hyperparameter controlling how many tokens will be replaced with MASK tokens during both the training and testing phases. In this work, we propose LogGPT, which leverages the GPT model to learn patterns in normal log sequences by predicting the next log entries in a sequence, and further proposes a novel reinforcement learning mechanism to enhance the performance for anomaly detection.

III. PRELIMINARY

In this section, we provide a detailed overview of two key components for log anomaly detection, log sequence preprocessing and log language model.

A. Log Sequence Preprocessing

The first step of log anomaly detection is to preprocess the log messages because it is hard to capture the sequential pattern from the raw text-based log messages. The major line of research in log anomaly detection is to first adopt a log parser, such as Drain [15], to extract the template from the

log messages, as shown in Figure 1. Each template usually indicates one type of log message, called a log key.

After getting the log keys, the sequence of raw log messages can be transformed into a sequence of log keys. In this case, the log keys are similar to the vocabulary in natural language, while the sequence is like a sentence consisting of a sequence of log keys. Therefore, a language model can be leveraged to model the log sequences.

Formally, after preprocessing, the log messages with the same template are represented by a log key $k \in \mathcal{K}$, where $\mathcal{K}$ indicates the set of log keys extracted from the log messages. Then, a log sequence is organized as ordered log keys, denoted as $S = \{k_1, \dots, k_t, \dots, k_T\}$, where T indicates the length of the log sequence.

B. Log Language Model

We use DeepLog [1] to illustrate the concept of the log language model. DeepLog leverages Long Short-Term Memory networks (LSTMs) for log language modeling. The primary objective of DeepLog is to learn a probabilistic model of normal execution from log data and then detect anomalies as significant deviations from normal patterns.

DeepLog is trained on $\mathcal{D} = \{S^i\}_{i=1}^N$ consisting of normal log sequences. The LSTM network in DeepLog is trained to predict the next log key in a sequence based on the preceding sequence. Formally, given a sequence of log keys $S_{1:T} = \{k_1, \dots, k_t, \dots, k_T\}$, where k_t indicates the log key at the t -th position. DeepLog trains an LSTM to model the conditional probability $p(k_{t+m+1}|S_{1:t+m})$ for $t = 1, 2, \dots, T - m - 1$, where m indicates the window size. Particularly, DeepLog adopts a sliding window with size m to split the sequences into a set of small windows and predict the next log key given the previous m log keys. The LSTM is trained to maximize the likelihood of the next log key given the preceding sequence, which can be formulated as the following objective function:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{T-m-1} \log p(k_{t+m+1}^i | S_{1:t+m}^i), \quad (1)$$

where θ denotes the parameters of LSTM.

During the anomaly detection phase, given a new sequence, DeepLog still splits the sequences into small windows and employs the trained LSTM model to predict the next log key. The LSTM model predicts a probability distribution over all possible log keys in $\mathcal{K}$, ranking them based on their likelihood of being the next key in the sequence. Then, an abnormal sequence will be labeled as abnormal if the observed log key does not appear in the Top-K prediction list multiple times across all sliding windows in that sequence.

The concept of Top-K predictions is introduced to account for the inherent uncertainty and variability in log sequences. Even in normal operations, there can be multiple valid “next” log keys as the systems usually have multiple normal patterns. Therefore, during the anomaly detection phase, instead of predicting a single ‘most likely’ next log key, the model identifies the Top-K most probable next log keys. As long as

the observed log key is in the Top-K list, we could consider the sequence normal.

The value of K , a tunable hyperparameter, determines the strictness of the model for anomaly detection. A smaller K results in a stricter model that allows fewer possibilities for the next log key, usually leading to high recall and low precision, while a larger K results in a more flexible model that considers a broader range of log keys as normal, usually resulting in high precision and low recall.

IV. LOGGPT

In this section, we introduce LogGPT, a novel log anomaly detection model based on GPT. Similar to DeepLog, LogGPT detects the log anomaly by examining whether the observed log key is in the Top-K prediction list. Because GPT is a more powerful structure compared to LSTM used by DeepLog, LogGPT does not need to further split the sequence into multiple small windows. Instead, LogGPT is trained to predict the next log key given the previous sequence, which intrinsically can capture the long-term dependence of log sequences. Moreover, besides leveraging the powerful GPT structure, we also propose a novel reinforcement learning strategy to further improve the performance of log anomaly detection.

The design of LogGPT is inspired by the training process of large language models, where the training process consists of two primary stages: pre-training and fine-tuning, as shown in Figure 2.

In the pre-training stage (Figure 2a), a generative log language model $f_\theta(\cdot)$ is trained on a corpus of normal log sequences $\mathcal{D}$, which allows the model to learn the underlying patterns and structures of normal system behavior. After pre-training, LogGPT is capable of generating log sequences based on a given part of the log sequences.

The fine-tuning stage (Figure 2b) is designed to further refine the model’s ability to distinguish between normal and abnormal log sequences. In this stage, we employ reinforcement learning techniques to finetune the pre-trained LogGPT. Borrowing the terminology from the large language model, we define a set of prompts $\mathcal{P} = \{S_{1:t}^i\}_{i=1}^N$, where $S_{1:t}^i \subseteq S_{1:T}^i$ and $S_{1:T}^i \in \mathcal{D}$. These prompts are fed into the LogGPT to generate the following sequence $\hat{S}_{t:T}^i$ step by step. We propose a novel reward, called the Top-K metric, to fine-tune LogGPT for anomaly detection.

A. Generative Log Language Model

LogGPT utilizes GPT-2 [16] for modeling the log sequences, which is based on Transformer decoder [12] that utilizes a self-attention mechanism to capture dependencies between log keys in the log sequence. LogGPT is trained to predict the next log key given the preceding log keys. The objective function for pretraining the LogGPT is defined as follows:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{T-1} \log p(k_{t+1}^i | S_{1:t}^i), \quad (2)$$

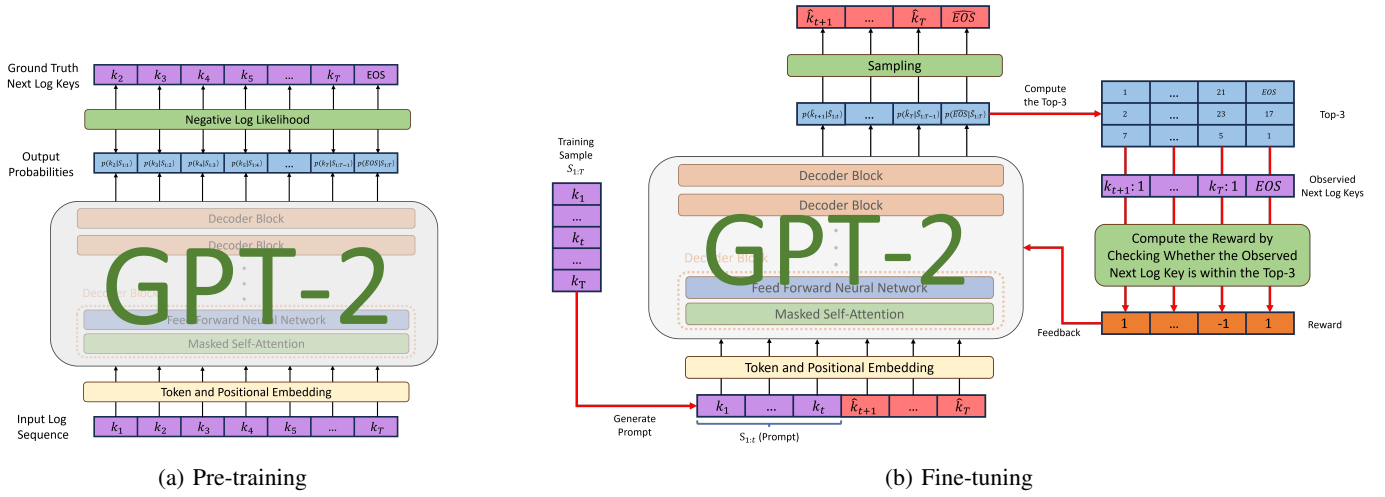


Fig. 2: Framework of LogGPT.

where θ denotes the parameters of LogGPT, N is the number of log sequences and T is the length of each sequence, $p(k_{t+1}^i | S_{1:t}^i)$ indicates the probability of log key at the $t+1$ -th position predicted by LogGPT given the sequence $S_{1:t}^i$.

Specifically, to derive $p(k_{t+1}^i | S_{1:t}^i)$, the structure of LogGPT can be defined as:

$$\mathbf{h}_t^i = \text{Transformer_Decoder}(S_{1:t}^i) \quad (3a)$$

$$p(k_{t+1}^i | S_{1:t}^i) = \text{Softmax}(\mathbf{h}_t^i \mathbf{W}), \quad (3b)$$

where $\mathbf{h}_t^i \in \mathbb{R}^d$ indicates the hidden representation derived from the Transformer decoder [12], [16], and $\mathbf{W} \in \mathbb{R}^{d \times |\mathcal{K}|}$ is the parameter of the language model head that maps the hidden representation to a probability distribution of all log keys in $\mathcal{K}$.

By training the model to predict the next log key in normal log sequences, LogGPT encodes the normal system behavior. After pre-training, GPT-2 is capable of generating a log sequence $\hat{S}_{t+1:T}^i = \{\hat{k}_{t+1}^i, \dots, \hat{k}_T^i\}$ based on a given part of the log sequence $S_{1:t}^i$. This capability is crucial for the subsequent fine-tuning stage, where the model is further refined to distinguish between normal and anomalous log sequences.

B. Reinforcement Learning for Log Anomaly Detection

In the context of LogGPT, we employ reinforcement learning to fine-tune the pre-trained GPT-2 model for the task of log anomaly detection. The reinforcement learning paradigm is particularly suitable for our task as it allows the model to learn from its predictions and adjust its behavior based on the feedback received, thereby enhancing its ability to detect anomalies. In the context of our framework, we define the following elements.

State: The state, denoted as $\tilde{S}_{1:t}^i = S_{1:t}^i$, is initially defined as the given part of a log sequence. As the model generates the log sequence $\hat{S}_{t+1:T}^i$ based on the given part, the state evolves dynamically. Specifically, for each step j where $t+1 \leq j \leq$

$T-1$, the state $\tilde{S}_{1:j}^i$ becomes the concatenation of the given part of the log sequence $S_{1:t}^i$ and the generated part of the log sequence $\hat{S}_{t+1:j}^i$, denoted as $\tilde{S}_{1:j}^i = \{S_{1:t}^i, \hat{S}_{t+1:j}^i\}$. The sequence $\tilde{S}_{1:j}^i$ is further transformed to a hidden representation $\tilde{\mathbf{h}}_j^i$ by the Transformer decoder shown in Equation 3a.

Action: An action is defined as sampling a log key from the $\mathcal{K}$ log keys with the highest probabilities predicted by LogGPT, denoted as $a_{j+1}^i \sim \text{Top-K}(p(\hat{k}_{j+1}^i | \tilde{S}_{1:j}^i))$.

Policy: A policy takes the form of LogGPT and is defined by its parameters. Specifically, given the current part of the sequence until the j -th position, the policy outputs a probability distribution over the action space, represented as $\pi_\theta(a_{j+1}^i | \tilde{\mathbf{h}}_j^i)$, where θ indicates the parameters of LogGPT.

Reward: The reward function provides feedback to the policy based on the quality of its actions. We propose a novel reward function to evaluate the predicted log key for anomaly detection, called the Top-K metric.

At each step, the Top-K metric checks whether the observed next log key is within the Top-K predicted log keys. If this is the case, the model receives a reward of 1; otherwise, it receives a reward of -1. Given a part of log sequence $S_{1:t}^i$, after an action is taken, the reward function is formulated as:

$$r_{j+1} = \begin{cases} 1, & \text{if } k_{j+1}^i \in \text{Top-K}(p(\hat{k}_{j+1}^i | \tilde{S}_{1:j}^i)) \\ -1, & \text{if } k_{j+1}^i \notin \text{Top-K}(p(\hat{k}_{j+1}^i | \tilde{S}_{1:j}^i)) \end{cases} \quad (4)$$

Here, k_{j+1}^i refers to the actual next log key, and $p(\hat{k}_{j+1}^i | \tilde{S}_{1:j}^i)$ denotes the probability distribution predicted by LogGPT over the action space given the current state.

The Top-K metric promotes better generalization and robustness of LogGPT in anomaly detection. By encouraging the model to predict a set of likely next log keys rather than a single most likely log key, the Top-K metric helps LogGPT learn a more nuanced representation of the normal log patterns. This approach recognizes that log data may contain inherent variability even for the normal log sequences, and a broader range of acceptable candidates can still reflect normal system

TABLE I: Statistics of the Datasets. The number in the parentheses indicates the unique log keys in the training set.

Dataset	# of Unique Log Keys	# of Log Sequences	Avg. Seq. Length	Training Data	Testing Data	
					Normal	Anomalous
HDFS	48 (15)	575,061	19	5,000	553,223	16,838
BGL	396 (160)	36,927	58	5,000	28,631	3,296
Thunderbird	7,703 (904)	112,959	166	5,000	67,039	40,920

behavior. The Top-K metric, therefore, enhances the precision of anomaly detection by aligning the model’s predictions with the complex nature of log data.

C. Policy Update

We adopt Proximal Policy Optimization (PPO) [17] for the policy update. PPO is a type of policy gradient method that optimizes the policy directly by maximizing the expected reward and can further maintain the stability of the learning process and prevent harmful updates. The objective function of PPO is defined as follows:

$$J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{i=1}^N \sum_{j=t}^{T-1} \frac{\pi_{\theta}(a_{j+1}^i | \mathbf{h}_j^i)}{\pi_{\theta_{\text{old}}}(a_{j+1}^i | \mathbf{h}_j^i)} r_{j+1} \right], \quad (5)$$

where π_{θ} is the new policy, $\pi_{\theta_{\text{old}}}$ is the old policy, and r_{j+1} is the reward for an action.

The policy π_{θ} is updated by performing gradient ascent on the objective function $J(\theta)$:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta), \quad (6)$$

where α is the learning rate.

The policy update process is repeated for a number of iterations until the policy converges or a maximum number of iterations is reached. The Top-K metric encourages the model to recognize the inherent variability in normal log data by rewarding predictions that include the actual next log key within a broader set.

D. Anomaly Detection

After fine-tuning, LogGPT is deployed to detect abnormal log sequences. Given a new log sequence $S_{1:T}$, LogGPT iteratively predicts the next log key k_{t+1} given the preceding subsequence $S_{1:t}$ for $1 \leq t \leq T-1$.

For each predicted log key, the model generates a set of Top-K predicted log keys. This set represents the K most likely log keys at the current position. The actual next log key is then compared to this set. As long as one actual log key is not in the set of Top-K predicted log keys, the whole log sequence will be flagged as anomalous.

V. EXPERIMENTS

A. Experimental Setup

Datasets. We evaluate LogGPT on three log datasets, namely HDFS, BGL, and Thunderbird. Table I shows the statistics of three datasets. For all the datasets, we randomly select 5000 normal log sequences as the training dataset.

- HDFS (Hadoop Distributed File System) [7]: This dataset is derived from Hadoop-based map-reduce jobs that were

run on Amazon EC2 nodes. The anomalies within this dataset are identified through a manual labeling process based on a set of predefined rules. The log sequences are constructed based on the session ID present in each log message, resulting in an average sequence length of 19. The HDFS dataset consists of 575,061 log sequences, out of which 16,838 have been labeled as anomalous.

- BGL (BlueGene/L Supercomputer System) [18]: The BGL dataset originates from a BlueGene/L supercomputer system, located at the Lawrence Livermore National Labs (LLNL). It includes both alert and non-alert messages, with the alert messages being treated as anomalies. Log sequences are formed using a time sliding window of 1 minute, yielding an average sequence length of 58. The BGL dataset contains 36,927 log sequences, with 3,296 of them classified as anomalous.
- Thunderbird [18]: This dataset is collected from another supercomputer system. The dataset used in this study comprises the first 20,000,000 log messages from the original Thunderbird dataset that compose 112,959 log sequences, with 40,920 of them marked as anomalous. Log sequences are created using a time sliding window of 1 minute, leading to an average sequence length of 166.

Baselines. We compare LogGPT with a variety of baseline methods, consisting of both traditional machine learning models and deep learning models:

- PCA (Principal Component Analysis) [19]: This technique constructs a counting matrix based on the frequency of log key sequences. It then reduces this matrix into a lower-dimensional space to identify anomalies.
- iForest (Isolation Forest) [8]: iForest is an unsupervised learning algorithm, which also adopts a counting matrix as input. It isolates anomalies instead of profiling normal data points. It represents features as tree structures and anomalies are detected as instances with short average path lengths on the constructed isolation trees.
- OCSVM (One-Class Support Vector Machine) [20]: OCSVM is a variant of the Support Vector Machine algorithm that is designed for anomaly detection tasks [9], [21]. The model is trained on normal data and finds the maximum margin hyperplane that separates the normal data from the origin.
- LogCluster [22]: LogCluster is a density-based log clustering approach that groups similar log messages together. Anomalies are detected as log messages that do not belong to any cluster or belong to small clusters.
- DeepLog [1]: DeepLog is a deep learning-based approach for anomaly detection in log data. It uses a long short-term memory (LSTM) network to model the log sequences and detect anomalies based on the prediction errors.
- LogAnomaly [10]: LogAnomaly models a log stream as a natural language sequence, which can detect both sequential and quantitative log anomalies simultaneously.

TABLE II: Experimental Results on HDFS, BGL, and Thunderbird Datasets.

Method	HDFS			BGL			Thunderbird		
	Precision	Recall	F-1 score	Precision	Recall	F-1 score	Precision	Recall	F-1 score
PCA	0.166 $\pm$ 0.008	0.059 $\pm$ 0.003	0.087 $\pm$ 0.002	0.117 $\pm$ 0.023	0.035 $\pm$ 0.007	0.054 $\pm$ 0.010	0.953 $\pm$ 0.004	0.980 $\pm$ 0.005	0.966 $\pm$ 0.003
iForest	0.043 $\pm$ 0.010	0.422 $\pm$ 0.224	0.078 $\pm$ 0.021	0.491 $\pm$ 0.364	0.037 $\pm$ 0.052	0.063 $\pm$ 0.090	0.338 $\pm$ 0.128	0.015 $\pm$ 0.011	0.028 $\pm$ 0.020
OCSVM	0.058 $\pm$ 0.012	0.910 $\pm$ 0.089	0.108 $\pm$ 0.021	0.073 $\pm$ 0.003	0.345 $\pm$ 0.010	0.121 $\pm$ 0.004	0.550 $\pm$ 0.004	0.998 $\pm$ 0.000	0.709 $\pm$ 0.003
LogCluster	0.996$\pm$0.003	0.368 $\pm$ 0.001	0.538 $\pm$ 0.001	0.941$\pm$0.015	0.641 $\pm$ 0.033	0.762 $\pm$ 0.021	0.977$\pm$0.005	0.291 $\pm$ 0.063	0.445 $\pm$ 0.067
DeepLog	0.793 $\pm$ 0.092	0.863 $\pm$ 0.031	0.824 $\pm$ 0.060	0.792 $\pm$ 0.048	0.946 $\pm$ 0.012	0.861 $\pm$ 0.028	0.864 $\pm$ 0.005	0.997 $\pm$ 0.000	0.926 $\pm$ 0.003
LogAnomaly	0.907 $\pm$ 0.017	0.369 $\pm$ 0.014	0.524 $\pm$ 0.017	0.884 $\pm$ 0.002	0.850 $\pm$ 0.009	0.867 $\pm$ 0.003	0.873 $\pm$ 0.005	0.996 $\pm$ 0.000	0.931 $\pm$ 0.003
OC4Seq	0.922 $\pm$ 0.059	0.758 $\pm$ 0.227	0.808 $\pm$ 0.157	0.441 $\pm$ 0.045	0.352 $\pm$ 0.044	0.391 $\pm$ 0.041	0.901 $\pm$ 0.046	0.823 $\pm$ 0.232	0.845 $\pm$ 0.177
LogBERT	0.754 $\pm$ 0.142	0.749 $\pm$ 0.037	0.745 $\pm$ 0.082	0.917 $\pm$ 0.006	0.892 $\pm$ 0.006	0.905 $\pm$ 0.005	0.962 $\pm$ 0.019	0.965 $\pm$ 0.008	0.963 $\pm$ 0.007
CAT	0.102 $\pm$ 0.022	0.422 $\pm$ 0.082	0.164 $\pm$ 0.034	0.177 $\pm$ 0.122	0.210 $\pm$ 0.184	0.190 $\pm$ 0.148	0.751 $\pm$ 0.072	0.516 $\pm$ 0.124	0.607 $\pm$ 0.120
LogGPT	0.884 $\pm$ 0.030	0.921$\pm$0.066	0.901$\pm$0.036	0.940 $\pm$ 0.010	0.977$\pm$0.018	0.958$\pm$0.011	0.973 $\pm$ 0.004	1.000$\pm$0.000	0.986$\pm$0.002

The asterisk indicates that LogGPT significantly outperforms the best baseline at the 0.05 level, according to the paired t-test.

TABLE III: Performance of LogGPT with or without reinforcement learning.

Metric	Approach	HDFS	BGL	Thunderbird
Precision	LogGPT w/o RL	0.932 $\pm$ 0.015	0.936 $\pm$ 0.011	0.971 $\pm$ 0.004
	LogGPT	0.884 $\pm$ 0.030	0.940 $\pm$ 0.010	0.973 $\pm$ 0.004
Recall	LogGPT w/o RL	0.790 $\pm$ 0.101	0.975 $\pm$ 0.018	1.000 $\pm$ 0.000
	LogGPT	0.921 $\pm$ 0.066	0.977 $\pm$ 0.018	1.000 $\pm$ 0.000
F-1 score	LogGPT w/o RL	0.853 $\pm$ 0.065	0.955 $\pm$ 0.010	0.985 $\pm$ 0.002
	LogGPT	0.901 $\pm$ 0.036	0.958 $\pm$ 0.011	0.986 $\pm$ 0.002

Significantly outperforms LogGPT w/o RL at the 0.05 level (paired t-test).

- OC4Seq (Multi-Scale One-Class Recurrent Neural Networks) [11]: OC4Seq is designed to detect anomalies in discrete event sequences. Recognizing that an anomalous sequence could be caused by individual events, subsequences of events, or the entire sequence, OC4Seq employs a multi-scale RNN framework to capture different levels of sequential patterns simultaneously.
- LogBERT [2]: LogBERT is a BERT-based architecture to capture the patterns of normal log sequences via a log language model. LogBERT is trained to predict the masked log keys on normal log sequences and detects the abnormal log sequences based on the prediction errors.
- CAT (Content-Aware Transformer) [14]: CAT is a self-attentive encoder-decoder transformer framework designed for anomaly detection in event sequences. It incorporates the semantic information of event content by using a content-awareness layer to generate representations of each event. The encoder learns preamble event sequence representations with content awareness, and the decoder embeds sequences under detection into a latent space where anomalies are distinguishable.

Implementation Details. We first employ Drain [15] to parse raw log messages into log keys. For the baseline models, we utilize the Loglizer [23] package to evaluate PCA, OCSVM, iForest, and LogCluster for anomaly detection. DeepLog and LogAnomaly are evaluated using the Deep-loglizer [24] package. For OC4Seq [1], LogBERT [2], and CAT [3], we use the open-source code provided by the authors separately.

As for LogGPT, we use a GPT model with 6 layers and 6 heads. The dimensions of the embeddings and hidden states are set to 60. The learning rate is set to 1e-4 for the pre-training

phase and 1e-6 for the fine-tuning phase. To accommodate different datasets, we set the K in Top-K to 50% of the training log keys. It means during the test phase if an observed log key is not in the top 50% of the prediction list from the GPT, the sequence will be labeled as an anomaly. This allows us to maintain a high level of flexibility when dealing with datasets of varying sizes and characteristics. The batch size for the pre-training phase is set to 16, and we train the model for 100 epochs. The episode is set to 20 with early stop criteria to prevent overfitting and ensure efficient training. The code for LogGPT is publicly available [4].

B. Experimental Results

Performance on Log Anomaly Detection. Table II illustrates the results and standard deviation of LogGPT and various baselines over 10 runs on the HDFS, BGL, and Thunderbird datasets. The asterisk in the table indicates that LogGPT significantly outperforms the best baseline for each dataset at the 0.05 level, according to the paired t-test.

First, we can observe that PCA, iForest, and OCSVM perform poorly on the HDFS and BGL datasets, as indicated by their low F-1 scores. However, PCA’s performance is notably better on the Thunderbird dataset, achieving a high F-1 score. This inconsistency in performance across datasets highlights the sensitivity of PCA to datasets.

LogCluster, specifically designed for log anomaly detection, shows improved performance over other traditional machine learning models, i.e., PCA, iForest, and OCSVM, on the HDFS and BGL datasets but is outperformed by PCA on the Thunderbird dataset. This pattern further emphasizes the importance of dataset-specific characteristics in determining the effectiveness of different methods.

Deep learning-based approaches, such as DeepLog, LogAnomaly, OC4seq, LogBERT, and CAT, outperform traditional methods across all three datasets, which shows the advantages of utilizing deep learning to capture complex patterns in log sequences.

Our proposed model, LogGPT, stands out by consistently achieving the highest F-1 scores across all three datasets, with significant margins over all baselines.

Ablation Studies. To investigate the contribution of reinforcement learning (RL) to the performance of LogGPT, we

¹<https://github.com/KnowledgeDiscovery/OC4Seq>

²<https://github.com/HelenGuohx/logbert>

³<https://github.com/mmichaelzhang/CAT>

⁴<https://github.com/nokia/LogGPT>

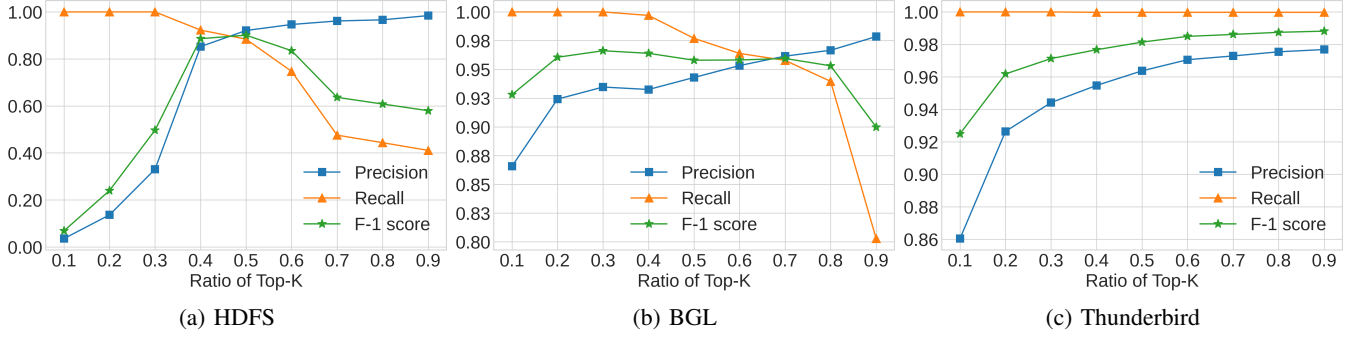


Fig. 3: Impact of the ratio of Top-K log keys.

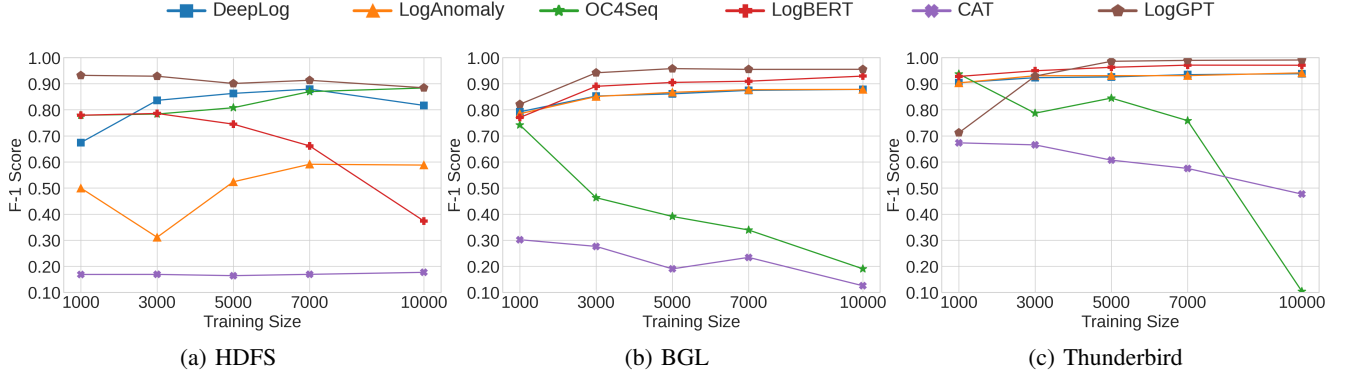


Fig. 4: Impact of the training size.

conducted an ablation study, comparing the performance of LogGPT with and without the RL component. The results are summarized in Table III.

First, we can notice that on both HDFFS and Thunderbird datasets, LogGPT significantly outperforms LogGPT without the RL component, which demonstrates that the RL component enhances the overall performance of LogGPT for log anomaly detection. Especially, on the HDFFS dataset, by finetuning the GPT model with RL reward, the recall achieved by LogGPT is improved with a large margin with a little sacrifice on precision, leading to extensive improvement in the F-1 score. It also shows that fine-tuning the log language model with Top-K reward can identify more log anomalies. Meanwhile, on the BGL dataset, we can also notice a slight improvement in F-1 of LogGPT compared to the one without the RL component. Another interesting finding is that even the LogGPT without the RL component already outperforms all baselines (shown in Table II) in three datasets, which also shows the advantage of leveraging the GPT model to capture the patterns of log sequences.

Parameter Analysis: Ratio of Top-K. LogGPT detects the anomalies by examining whether the observed log key is in the Top-K list predicted by GPT. Therefore, K is an important parameter to determine the anomalies. We first analyze the difference in the performance by tuning K for anomaly detection. By default, K is set as 50% of unique log

keys. It means if the next log key falls into the top 50% of unique log keys predicted by GPT, the sequence is normal.

The impact of different top-K ratios on the precision, recall, and F-1 score for the HDFFS, BGL, and Thunderbird datasets is illustrated in Figure 3. On both HDFFS and BGL datasets, we have similar observations. With the increasing of ratios as normal log keys, the recall keeps decreasing when the ratio is greater than a threshold, such as 40% in HDFFS and BGL. This happens because when we have a large ratio, most of the keys are considered normal. In this case, the recall will be low. On the other hand, if the observed log key is predicted with an extremely low probability at a specific position, with a high chance, this log key is abnormal. Therefore, we can observe the increase in precision along with the increase in ratios.

For the Thunderbird dataset, the precision increases as the top-K ratio increases, while the recall remains almost constant, with a slight decrease at higher top-K ratios. The F-1 score increases steadily, reaching a peak at a specific top-K ratio. The reason for this behavior can be attributed to the inherent characteristics of the Thunderbird dataset. It is likely that the normal data within the Thunderbird dataset has high variability, which needs a broader range of acceptable continuations in the log sequences to reduce the false positive. As the top-K ratio increases, LogGPT becomes more selective in flagging anomalies, thereby increasing precision by reducing false positives.

Overall, a low top-K ratio tends to lead to high recall but low precision, while a high top-K ratio leads to high precision but potentially lower recall. The optimal top-K ratio varies across datasets, reflecting the unique characteristics of each dataset.

Scalability Analysis: Training Size. It is well known that deep learning models usually require a sufficient number of training samples. The impact of training size on the performance of log anomaly detection models is critical. By analyzing the F-1 scores of various models across different training sizes, we can gain insights into their effectiveness and efficiency. In this experiment, we compare LogGPT with other deep learning-based baselines, across three datasets by varying the training size. Figure 4 shows the experimental results.

The effect of the training size on the HDFS dataset reveals distinct patterns across different models (shown in Figure 4a). LogGPT demonstrates consistent performance across various training sizes, highlighting its robustness and ability to generalize well. OC4Seq shows a consistent increase in performance with the training size, indicating that it benefits from more extensive training data. DeepLog and LogAnomaly exhibit fluctuations in performance, which may be attributed to the sensitivity to training size. The decline in performance for LogBERT and stability for CAT may reflect limitations in their ability to leverage additional training data without changing other hyper-parameters. The varying behaviors of these models underscore the importance of carefully selecting the training size based on the model's characteristics.

We have similar observations on BGL and Thunderbird datasets. First, with larger training sizes, the performance of LogGPT, DeepLog, LogAnomaly, and LogBERT keep improving, which shows that these models can benefit from additional training data. Meanwhile, LogGPT can outperform those baselines in most cases. However, the sharp decline for OC4Seq and overall downward trend for CAT may indicate overfitting or challenges in generalizing from larger training sets.

Overall, LogGPT can achieve very good performance in three datasets. More training samples can further boost the performance of LogGPT.

VI. CONCLUSION

In this work, we introduced LogGPT, a novel approach to log anomaly detection that builds upon GPT models, further enhanced by a reinforcement learning strategy. Through modeling log sequences as natural language, LogGPT innovatively adapts GPT for log anomaly detection. More importantly, recognizing the existing gap between language modeling and anomaly detection, LogGPT integrates a fine-tuning process guided by a novel Top-K reward metric for anomaly detection. Extensive experiments conducted across various datasets demonstrated the effectiveness of LogGPT, showcasing significant improvements over existing state-of-the-art methods.

REFERENCES

- [1] M. Du, F. Li, G. Zheng, and V. Srikumar, "Deeplog: Anomaly detection and diagnosis from system logs through deep learning," in *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, 2017, pp. 1285–1298.
- [2] H. Guo, S. Yuan, and X. Wu, "Logbert: Log anomaly detection via bert," in *2021 international joint conference on neural networks (IJCNN)*. IEEE, 2021, pp. 1–8.
- [3] G. Pang, C. Shen, L. Cao, and A. V. D. Hengel, "Deep learning for anomaly detection: A review," *ACM computing surveys (CSUR)*, vol. 54, no. 2, pp. 1–38, 2021.
- [4] V.-H. Le and H. Zhang, "Log-based anomaly detection with deep learning: How far are we?" in *Proceedings of the 44th international conference on software engineering*, 2022, pp. 1356–1367.
- [5] R. Chalapathy and S. Chawla, "Deep learning for anomaly detection: A survey," *arXiv preprint arXiv:1901.03407*, 2019.
- [6] M. Landauer, S. Onder, F. Skopik, and M. Wurzenberger, "Deep learning for anomaly detection in log data: A survey," *Machine Learning with Applications*, vol. 12, p. 100470, 2023.
- [7] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, 2009, pp. 117–132.
- [8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *2008 eighth IEEE international conference on data mining*. IEEE, 2008, pp. 413–422.
- [9] Y. Wang, J. Wong, and A. Miner, "Anomaly intrusion detection using one class svm," in *Proceedings from the Fifth Annual IEEE SMC Information Assurance Workshop*, 2004. IEEE, 2004, pp. 358–364.
- [10] W. Meng, Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, Y. Chen, R. Zhang, S. Tao, P. Sun *et al.*, "Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *IJCAI*, vol. 19, no. 7, 2019, pp. 4739–4745.
- [11] Z. Wang, Z. Chen, J. Ni, H. Liu, H. Chen, and J. Tang, "Multi-scale one-class recurrent neural networks for discrete event sequence anomaly detection," in *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*, 2021, pp. 3726–3734.
- [12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [14] S. Zhang, Y. Liu, X. Zhang, W. Cheng, H. Chen, and H. Xiong, "Cat: Beyond efficient transformer for content-aware anomaly detection in event sequences," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022, pp. 4541–4550.
- [15] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *2017 IEEE international conference on web services (ICWS)*. IEEE, 2017, pp. 33–40.
- [16] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," 2019.
- [17] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [18] A. Oliner and J. Stearley, "What supercomputers say: A study of five system logs," in *37th annual IEEE/IFIP international conference on dependable systems and networks (DSN'07)*. IEEE, 2007, pp. 575–584.
- [19] W. Xu, L. Huang, A. Fox, D. Patterson, and M. Jordan, "Largescale system problem detection by mining console logs," *Proceedings of SOSP'09*, 2009.
- [20] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural computation*, vol. 13, no. 7, pp. 1443–1471, 2001.
- [21] K.-L. Li, H.-K. Huang, S.-F. Tian, and W. Xu, "Improving one-class svm for anomaly detection," in *Proceedings of the 2003 international conference on machine learning and cybernetics (IEEE Cat. No. 03EX693)*, vol. 5. IEEE, 2003, pp. 3077–3081.
- [22] Q. Lin, H. Zhang, J.-G. Lou, Y. Zhang, and X. Chen, "Log clustering based problem identification for online service systems," in *Proceedings of the 38th International Conference on Software Engineering Companion*, 2016, pp. 102–111.
- [23] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience report: System log analysis for anomaly detection," in *2016 IEEE 27th international symposium on software reliability engineering (ISSRE)*. IEEE, 2016, pp. 207–218.

- [24] Z. Chen, J. Liu, W. Gu, Y. Su, and M. R. Lyu, “Experience report: Deep learning-based system log analysis for anomaly detection,” *arXiv preprint arXiv:2107.05908*, 2021.